```
import numpy as np
import matplotlib.pyplot as plt
import os, shutil
from collections import Counter
import copy
import time
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, models, transforms
from torchvision.transforms.functional import to_pil_image
from torchvision.utils import make_grid

import warnings
warnings.filterwarnings("ignore")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

Loading Dataset

```
INTERPOLATION_MODE = transforms.InterpolationMode.BICUBIC
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]


def transform_image_( IMG_SIZE=384):
    """
        Our image size is 224x224 it needs to be converted to 384x384
    """
    train_transform_aug_384 = transforms.Compose([
        transforms.Resize((IMG_SIZE, IMG_SIZE),
interpolation=INTERPOLATION_MODE),
        transforms.RandomRotation(40),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
    ])

    return train_transform_aug_384

transform_image = transform_image_(384)
```

```
train_dir = "/kaggle/input/railway-track-fault-detection-resized-224-
x-224/"
train_dataset = datasets.ImageFolder(f"{train_dir}train",
transform=transform_image)
test_dataset = datasets.ImageFolder(f"{train_dir}test",
transform=transform_image)
val_dataset = datasets.ImageFolder(f"{train_dir}valid",
transform=transform_image)

print(f"Train Dataset: {len(train_dataset)}")
print(f"test Dataset: {len(test_dataset)}")
print(f"val Dataset: {len(val_dataset)}")
```

```
Train Dataset: 300
test Dataset: 22
val Dataset: 62
```

```
print("Balanced Dataset")
print("Train Dataset")
for cls, idx in train_dataset.class_to_idx.items():
    print(f"{cls}: {sum(1 for label in train_dataset.targets if label
== idx)}")

print("\n Test Dataset")
for cls, idx in test_dataset.class_to_idx.items():
    print(f"{cls}: {sum(1 for label in test_dataset.targets if label
== idx)}")

print("\n Val Dataset")
for cls, idx in val_dataset.class_to_idx.items():
    print(f"{cls}: {sum(1 for label in val_dataset.targets if label ==
idx)}")
```

```
Balanced Dataset
Train Dataset
Defective: 150
Non defective: 150

 Test Dataset
Defective: 11
Non defective: 11

 Val Dataset
Defective: 31
Non defective: 31
```

Showing Augmented Images

```
def unnormalize(tensor, mean=IMAGENET_MEAN, std=IMAGENET_STD):
    """Un-normalizes a tensor for display."""
    # Clone to avoid modifying the original tensor
```

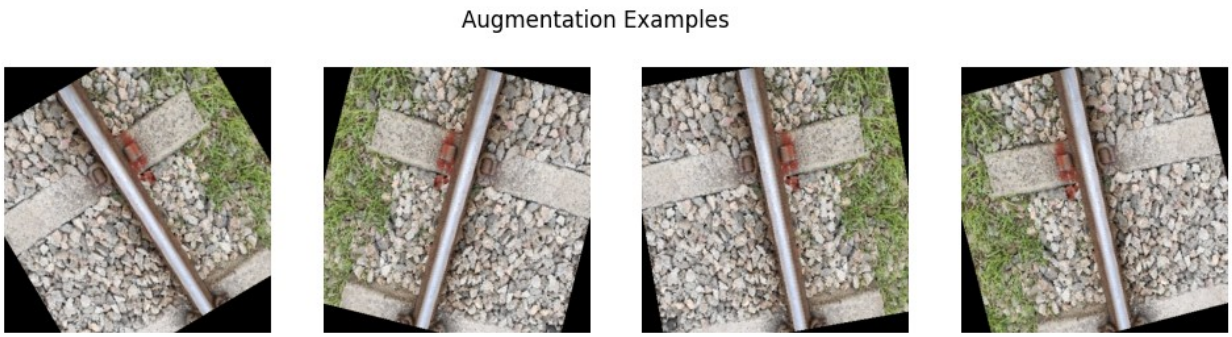
```
    tensor = tensor.clone()

    mean = torch.tensor(mean).view(3, 1, 1)
    std = torch.tensor(std).view(3, 1, 1)
    tensor.mul_(std).add_(mean) # tensor * std + mean
    tensor = torch.clamp(tensor, 0, 1)
    return tensor

sample_img_path = "/kaggle/input/railway-track-fault-detection-
resized-224-x-224/train/Defective/
131004959_171473721383413_8222187747086021011_n.jpg"
img = Image.open(sample_img_path).convert('RGB')
plt.figure(figsize=(12, 3))
for i in range(4):
    aug_img = transform_image(img) # Apply the full transform

    # We must un-normalize and permute to plot
    aug_img = unnormalize(aug_img).permute(1, 2, 0)

    plt.subplot(1, 4, i + 1)
    plt.imshow(aug_img)
    plt.axis('off')
plt.suptitle("Augmentation Examples")
plt.show()
```




```
from torchvision.utils import save_image

# Apply Augmentation in our dataset
def augment_dataset(image_transformer, output_dir, dataset,
nu_new_images=2):
    os.makedirs(output_dir, exist_ok=True)

    for img_path, label in tqdm(dataset.samples):
        img = Image.open(img_path).convert("RGB")

        class_name = dataset.classes[label]
        class_out_dir = os.path.join(output_dir, class_name)
        os.makedirs(class_out_dir, exist_ok=True)
```

```
        # 1 save original copy
        base_name = os.path.basename(img_path).split('.')[0]
        img.save(os.path.join(class_out_dir, f"{base_name}.png"))

        # 2 save 'copies' augmented versions
        for i in range(nu_new_images):
            aug_img = image_transformer(img)
            save_image(aug_img, os.path.join(class_out_dir,
f"{base_name}_aug{i}.png"))

    print(f"New Image dataset is saved at {output_dir}")
    new_dataset = datasets.ImageFolder(f"{output_dir}",
transform=image_transformer)
    return new_dataset

train_dataset_ = augment_dataset(transform_image,
"/kaggle/working/training", train_dataset)
```

```
100%|██████████| 300/300 [00:29<00:00, 10.18it/s]

New Image dataset is saved at /kaggle/working/training
```

```
val_dataset_ = augment_dataset(transform_image,
"/kaggle/working/validation", val_dataset)
test_dataset_ = augment_dataset(transform_image,
"/kaggle/working/testing", test_dataset)
```

```
100%|██████████| 62/62 [00:06<00:00,  9.70it/s]

New Image dataset is saved at /kaggle/working/validation

100%|██████████| 22/22 [00:02<00:00, 10.03it/s]

New Image dataset is saved at /kaggle/working/testing
```

```
print("Balanced Dataset_")
print("Train Dataset_")
for cls, idx in train_dataset_.class_to_idx.items():
    print(f"{cls}: {sum(1 for label in train_dataset_.targets if label
== idx)}")

print("\n Test Dataset")
for cls, idx in test_dataset_.class_to_idx.items():
    print(f"{cls}: {sum(1 for label in test_dataset_.targets if label
== idx)}")

print("\n Val Dataset_")
for cls, idx in val_dataset_.class_to_idx.items():
```

```
    print(f"{cls}: {sum(1 for label in val_dataset_.targets if label
== idx)}")
```

```
Balanced Dataset_
Train Dataset_
Defective: 447
Non defective: 450

 Test Dataset
Defective: 33
Non defective: 33

 Val Dataset_
Defective: 93
Non defective: 93
```

Training Utils

```
# Defining Some functions for training and testing

class EarlyStopper:
    """
    A simple early stopper.
    """
    def __init__(self, patience=5, min_delta=0, mode='max'):
        self.patience = patience
        self.min_delta = min_delta
        self.mode = mode
        self.counter = 0
        self.best_metric = -np.inf if mode == 'max' else np.inf
        self.best_model_weights = None
        self.should_stop = False

    def check(self, metric, model):
        """Checks if training should stop."""
        is_better = False
        if self.mode == 'max':
            if metric > self.best_metric + self.min_delta:
                is_better = True
        else: # min mode
            if metric < self.best_metric - self.min_delta:
                is_better = True

        if is_better:
            self.best_metric = metric
            self.counter = 0
            # Save the best model weights
            self.best_model_weights =
```

```
copy.deepcopy(model.state_dict())
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True

        return self.should_stop

def train_model(model, train_loader, val_loader, criterion, optimizer,
device, epochs=50, patience=5, verbose=1):
    """
    Reusable function to train and validate a PyTorch model.
    """
    history = {
        'train_loss': [], 'train_acc': [],
        'val_loss': [], 'val_acc': []
    }

    # Use 'val_acc' in 'max' mode for early stopping
    early_stopper = EarlyStopper(patience=patience, mode='max')

    start_time = time.time()

    for epoch in range(epochs):
        # --- Training Phase ---
        model.train()
        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            # Zero the parameter gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            # Backward pass and optimize
            loss.backward()
            optimizer.step()

            # Statistics
            model.eval()
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)
            model.train()
```

```
        epoch_train_loss = running_loss / len(train_loader.dataset)
        epoch_train_acc = running_corrects.double() /
len(train_loader.dataset)

        # --- Validation Phase ---
        model.eval()
        running_loss = 0.0
        running_corrects = 0

        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)

                outputs = model(inputs)
                loss = criterion(outputs, labels)

                _, preds = torch.max(outputs, 1)
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

        epoch_val_loss = running_loss / len(val_loader.dataset)
        epoch_val_acc = running_corrects.double() /
len(val_loader.dataset)

        # Log history
        history['train_loss'].append(epoch_train_loss)
        history['train_acc'].append(epoch_train_acc.item())
        history['val_loss'].append(epoch_val_loss)
        history['val_acc'].append(epoch_val_acc.item())

        if verbose:
            print(f"Epoch {epoch+1}/{epochs} | "
                f"Train Loss: {epoch_train_loss:.4f} Acc:
{epoch_train_acc:.4f} | "
                f"Val Loss: {epoch_val_loss:.4f} Acc:
{epoch_val_acc:.4f}")

        # Check early stopping
        if early_stopper.check(epoch_val_acc, model):
            print(f"Early stopping triggered after {epoch+1} epochs.")
            break

    # Load best model weights
    if early_stopper.best_model_weights:
        model.load_state_dict(early_stopper.best_model_weights)

    time_elapsed = time.time() - start_time
    print(f"\nTraining complete in {time_elapsed // 60:.0f}m
{time_elapsed % 60:.0f}s")
```

```
    print(f"Best Val Acc: {early_stopper.best_metric:.4f}")

    return model, history

from sklearn.metrics import precision_score, recall_score, f1_score

def evaluate_model(model, loader, criterion, device):
    """Evaluates the model and computes precision, recall, F1 for
multi-class classification."""
    model.eval()
    running_loss = 0.0
    running_corrects = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds == labels.data)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    total_loss = running_loss / len(loader.dataset)
    total_acc = running_corrects.double() / len(loader.dataset)
    precision = precision_score(all_labels, all_preds,
average='macro')
    recall = recall_score(all_labels, all_preds, average='macro')
    f1 = f1_score(all_labels, all_preds, average='macro')

    print(f"Loss: {total_loss:.4f}")
    print(f"Accuracy: {total_acc:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-score: {f1:.4f}")

    return total_loss, total_acc.item(), precision, recall, f1
```

Simple CNN

```
class SimpleCNN(nn.Module):
    """
```

```
    A simple CNN for classification.
    """
    def __init__(self, in_channels, num_classes):
        """
        Args:
            in_channels (int): Number of input channels (e.g., 3 for
RGB, 1 for grayscale).
            num_classes (int): Number of output classes.
        """
        super(SimpleCNN, self).__init__()

        # --- Convolutional Block 1 ---
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=3,
padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2) # Halves
spatial dimensions

        # --- Convolutional Block 2 ---
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2) # Halves
spatial dimensions again

        # --- Adaptive Pooling Layer ---
        # This layer ensures the output spatial dimension is always
7x7,
        # regardless of the input image size. This makes the linear
layer input consistent.
        self.adaptive_pool = nn.AdaptiveAvgPool2d((7, 7))

        # --- Classifier Head ---
        self.flatten = nn.Flatten()

        # Input features to the linear layer will be 64 (from conv2) *
7 * 7
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        # Pass through conv blocks
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))

        # Pass through adaptive pooling
        x = self.adaptive_pool(x)

        # Flatten and pass through classifier
```

```
        x = self.flatten(x)
        x = self.relu3(self.fc1(x))
        x = self.fc2(x) # Output logits (CrossEntropyLoss applies
softmax)
        return x

def run_training_pipeline(train_dataset_, test_dataset_, val_dataset_,
in_channels, num_classes, device=device):
    """
    A complete function to set up and run the training and evaluation
pipeline.

    Args:
        train_dataset: The full (already augmented) training dataset.
        test_dataset: A separate dataset for final evaluation.
        in_channels (int): Number of input image channels (e.g., 3 for
RGB).
        num_classes (int): Number of output classes.
    """

    # --- 1. Define Hyperparameters ---
    BATCH_SIZE = 32
    LEARNING_RATE = 0.001
    MAX_EPOCHS = 100       # Max epochs to run
    PATIENCE = 10          # Patience for your EarlyStopper
    VAL_SPLIT_RATIO = 0.2  # 20% of training data for validation

    train_loader = DataLoader(train_dataset_, batch_size=BATCH_SIZE,
shuffle=True, pin_memory=True)
    val_loader = DataLoader(val_dataset_, batch_size=BATCH_SIZE,
shuffle=False, pin_memory=True)
    test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
shuffle=False, pin_memory=True)

    # --- 4. Initialize Model, Criterion, Optimizer ---
    model = SimpleCNN(in_channels=in_channels,
num_classes=num_classes).to(device)

    # Loss function (standard for multi-class classification)
    criterion = nn.CrossEntropyLoss()

    # Optimizer (Adam is a good default)
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

    # --- 5. Run Training (using your function) ---
    print("\nStarting Training...")
    model, history = train_model(
        model=model,
        train_loader=train_loader,
        val_loader=val_loader,
```

```
        criterion=criterion,
        optimizer=optimizer,
        device=device,
        epochs=MAX_EPOCHS,
        patience=PATIENCE
    )

    # --- 6. Evaluate on Test Set (using your function) ---
    print("\nEvaluating on Test Set...")
    print("-------------------------")
    evaluate_model(
        model=model,
        loader=test_loader,
        criterion=criterion,
        device=device
    )

    print("\nPipeline finished.")
    return model, history

device
```

```
device(type='cuda')
```

```
model, history = run_training_pipeline(train_dataset_, test_dataset_,
val_dataset_, 3, len(train_dataset.classes), device=device)
```

```
Starting Training...
Epoch 1/100 | Train Loss: 0.6781 Acc: 0.6310 | Val Loss: 0.6234 Acc:
0.6237
Epoch 2/100 | Train Loss: 0.6193 Acc: 0.6734 | Val Loss: 0.5867 Acc:
0.6720
Epoch 3/100 | Train Loss: 0.5784 Acc: 0.7280 | Val Loss: 0.5715 Acc:
0.6774
Epoch 4/100 | Train Loss: 0.5673 Acc: 0.7280 | Val Loss: 0.4759 Acc:
0.8011
Epoch 5/100 | Train Loss: 0.5740 Acc: 0.7213 | Val Loss: 0.4880 Acc:
0.7742
Epoch 6/100 | Train Loss: 0.5336 Acc: 0.7525 | Val Loss: 0.4557 Acc:
0.8065
Epoch 7/100 | Train Loss: 0.5359 Acc: 0.7313 | Val Loss: 0.5017 Acc:
0.7527
Epoch 8/100 | Train Loss: 0.4998 Acc: 0.7570 | Val Loss: 0.4949 Acc:
0.7688
Epoch 9/100 | Train Loss: 0.4904 Acc: 0.7703 | Val Loss: 0.5080 Acc:
0.7742
Epoch 10/100 | Train Loss: 0.4680 Acc: 0.7726 | Val Loss: 0.4125 Acc:
0.8226
Epoch 11/100 | Train Loss: 0.4573 Acc: 0.8004 | Val Loss: 0.4062 Acc:
```

```
0.8118
Epoch 12/100 | Train Loss: 0.4318 Acc: 0.8038 | Val Loss: 0.3840 Acc:
0.8333
Epoch 13/100 | Train Loss: 0.4419 Acc: 0.8094 | Val Loss: 0.4496 Acc:
0.8065
Epoch 14/100 | Train Loss: 0.4882 Acc: 0.7570 | Val Loss: 0.4001 Acc:
0.7849
Epoch 15/100 | Train Loss: 0.4176 Acc: 0.8116 | Val Loss: 0.3542 Acc:
0.8280
Epoch 16/100 | Train Loss: 0.3959 Acc: 0.8060 | Val Loss: 0.5701 Acc:
0.7419
Epoch 17/100 | Train Loss: 0.5063 Acc: 0.7659 | Val Loss: 0.4281 Acc:
0.7796
Epoch 18/100 | Train Loss: 0.4555 Acc: 0.7781 | Val Loss: 0.4254 Acc:
0.8333
Epoch 19/100 | Train Loss: 0.4479 Acc: 0.7770 | Val Loss: 0.3799 Acc:
0.8280
Epoch 20/100 | Train Loss: 0.3964 Acc: 0.8071 | Val Loss: 0.3653 Acc:
0.8602
Epoch 21/100 | Train Loss: 0.3780 Acc: 0.8328 | Val Loss: 0.3772 Acc:
0.8280
Epoch 22/100 | Train Loss: 0.3866 Acc: 0.8261 | Val Loss: 0.3712 Acc:
0.8387
Epoch 23/100 | Train Loss: 0.3662 Acc: 0.8450 | Val Loss: 0.3844 Acc:
0.8656
Epoch 24/100 | Train Loss: 0.3517 Acc: 0.8495 | Val Loss: 0.4569 Acc:
0.8011
Epoch 25/100 | Train Loss: 0.3491 Acc: 0.8339 | Val Loss: 0.3891 Acc:
0.8495
Epoch 26/100 | Train Loss: 0.3505 Acc: 0.8339 | Val Loss: 0.4127 Acc:
0.8548
Epoch 27/100 | Train Loss: 0.3446 Acc: 0.8439 | Val Loss: 0.4387 Acc:
0.8118
Epoch 28/100 | Train Loss: 0.3245 Acc: 0.8473 | Val Loss: 0.3548 Acc:
0.8656
Epoch 29/100 | Train Loss: 0.3358 Acc: 0.8562 | Val Loss: 0.4461 Acc:
0.8280
Epoch 30/100 | Train Loss: 0.3103 Acc: 0.8707 | Val Loss: 0.3953 Acc:
0.8710
Epoch 31/100 | Train Loss: 0.3270 Acc: 0.8473 | Val Loss: 0.5216 Acc:
0.8172
Epoch 32/100 | Train Loss: 0.3003 Acc: 0.8785 | Val Loss: 0.4269 Acc:
0.8602
Epoch 33/100 | Train Loss: 0.2791 Acc: 0.8852 | Val Loss: 0.4869 Acc:
0.8441
Epoch 34/100 | Train Loss: 0.3166 Acc: 0.8562 | Val Loss: 0.4573 Acc:
0.8387
Epoch 35/100 | Train Loss: 0.3108 Acc: 0.8696 | Val Loss: 0.3896 Acc:
0.8656
```

```
Epoch 36/100 | Train Loss: 0.2645 Acc: 0.8974 | Val Loss: 0.5523 Acc: 0.8118
Epoch 37/100 | Train Loss: 0.2572 Acc: 0.8997 | Val Loss: 0.5059 Acc: 0.8226
Epoch 38/100 | Train Loss: 0.2480 Acc: 0.8941 | Val Loss: 0.4749 Acc: 0.8495
Epoch 39/100 | Train Loss: 0.2263 Acc: 0.9130 | Val Loss: 0.4296 Acc: 0.8817
Epoch 40/100 | Train Loss: 0.2447 Acc: 0.9075 | Val Loss: 0.5127 Acc: 0.8172
Epoch 41/100 | Train Loss: 0.2436 Acc: 0.9052 | Val Loss: 0.5746 Acc: 0.8333
Epoch 42/100 | Train Loss: 0.2127 Acc: 0.9231 | Val Loss: 0.4833 Acc: 0.8710
Epoch 43/100 | Train Loss: 0.2226 Acc: 0.9231 | Val Loss: 0.4258 Acc: 0.8602
Epoch 44/100 | Train Loss: 0.2271 Acc: 0.9175 | Val Loss: 0.5841 Acc: 0.8548
Epoch 45/100 | Train Loss: 0.2057 Acc: 0.9208 | Val Loss: 0.5216 Acc: 0.8387
Epoch 46/100 | Train Loss: 0.1855 Acc: 0.9287 | Val Loss: 0.6371 Acc: 0.8118
Epoch 47/100 | Train Loss: 0.2165 Acc: 0.9342 | Val Loss: 0.7575 Acc: 0.8011
Epoch 48/100 | Train Loss: 0.2578 Acc: 0.9097 | Val Loss: 0.5368 Acc: 0.8226
Epoch 49/100 | Train Loss: 0.1923 Acc: 0.9287 | Val Loss: 0.5479 Acc: 0.8548
Early stopping triggered after 49 epochs.

Training complete in 8m 44s
Best Val Acc: 0.8817

Evaluating on Test Set...
--------------------------
Loss: 0.5575
Accuracy: 0.6818
Precision: 0.6964
Recall: 0.6818
F1-score: 0.6758

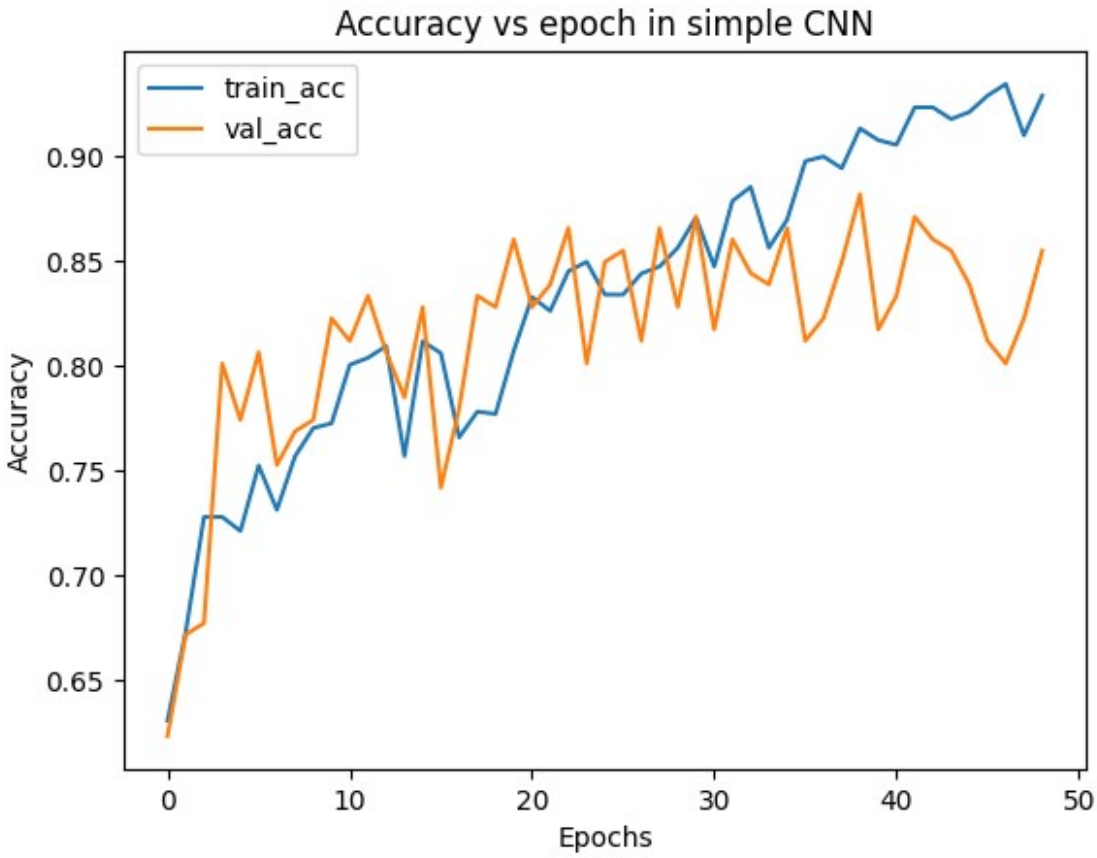
Pipeline finished.
```

```
history.keys()
```

```
dict_keys(['train_loss', 'train_acc', 'val_loss', 'val_acc'])
```

```
plt.plot(history['train_acc'], label="train_acc")
plt.plot(history['val_acc'], label="val_acc")
plt.title("Accuracy vs epoch in simple CNN")
```

```
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



Hyper Parameter Tuning on simple CNN

```
class TunableCNN(nn.Module):
    """
    A CNN that accepts hyperparameters for tuning.
    Can be initialized by an Optuna trial OR by passing parameters
directly.
    """
    def __init__(self, in_channels, num_classes, trial=None,
**params):
        super(TunableCNN, self).__init__()

        # --- Get Hyperparameters ---
        if trial:
            # We are in an Optuna trial, suggest parameters
            out_channels_1 =
```

```
trial.suggest_categorical("out_channels_1", [16, 32, 64])
            out_channels_2 =
trial.suggest_categorical("out_channels_2", [64, 128])
            dropout_rate = trial.suggest_float("dropout_rate", 0.1,
0.5)
            fc_features = trial.suggest_categorical("fc_features",
[64, 128, 256])
        else:
            # We are creating the final model, use passed-in params
            out_channels_1 = params['out_channels_1']
            out_channels_2 = params['out_channels_2']
            dropout_rate = params['dropout_rate']
            fc_features = params['fc_features']

        # --- Build Model (same as before) ---
        self.conv1 = nn.Conv2d(in_channels, out_channels_1,
kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(out_channels_1, out_channels_2,
kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.adaptive_pool = nn.AdaptiveAvgPool2d((7, 7))
        self.flatten = nn.Flatten()

        self.fc1 = nn.Linear(out_channels_2 * 7 * 7, fc_features)
        self.relu3 = nn.ReLU()
        self.dropout = nn.Dropout(dropout_rate)
        self.fc2 = nn.Linear(fc_features, num_classes)

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = self.adaptive_pool(x)
        x = self.flatten(x)
        x = self.relu3(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

import optuna
import torch
import torch.optim as optim
from torch.utils.data import DataLoader, random_split

in_channels = 3
num_classes = 2
```

```
def objective(trial):
    """
    The Optuna objective function.
    """
    try:
        # --- 1. Define Hyperparameter Search Space ---

        # Optimizer and Learning Rate
        lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
        optimizer_name = trial.suggest_categorical("optimizer",
["Adam", "RMSprop", "SGD"])

        # Data and Training
        batch_size = trial.suggest_categorical("batch_size", [16, 32,
64])

        # Model parameters are suggested inside TunableCNN's __init__

        # --- 2. Setup (Data, Model, Optimizer) ---
        train_loader = DataLoader(train_dataset_,
batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset_, batch_size=batch_size,
shuffle=False)
        test_loader = DataLoader(test_dataset_, batch_size=batch_size,
shuffle=False)

        # Initialize the model using the trial
        model = TunableCNN(in_channels, num_classes, trial).to(device)

        # Loss function
        criterion = nn.CrossEntropyLoss()

        # Optimizer
        optimizer_class = getattr(optim, optimizer_name)
        optimizer = optimizer_class(model.parameters(), lr=lr)

        # --- 3. Run Training (using your function) ---
        # We set verbose=0 to avoid flooding the console during HPO

        # Set fixed epochs/patience for the tuning run
        epochs = 50
        patience = 5 # Early stopping patience

        model, history = train_model(
            model=model,
            train_loader=train_loader,
            val_loader=val_loader,
            criterion=criterion,
            optimizer=optimizer,
```

```
            device=device,
            epochs=epochs,
            patience=patience,
            verbose=0  # Set to 0 to keep Optuna's output clean
        )

        # --- 4. Return the metric to optimize ---
        best_val_acc = max(history['val_acc'])
        return best_val_acc

    except Exception as e:
        # Handle potential errors during a trial (e.g., OOM)
        print(f"Trial failed with error: {e}")
        return 0.0 # Return a bad score

# --- 2. Create the Study ---
# We want to maximize the validation accuracy
study = optuna.create_study(direction="maximize")

# --- 3. Run Optimization ---
# We use a lambda to pass the extra, fixed arguments to the objective
n_trials = 10 # Run 50 different trials
print(f"Starting Optuna study with {n_trials} trials...")

# Use your *full* training dataset (it will be split inside the
objective)
# !! Make sure `train_dataset_`, `IN_CHANNELS`, `NUM_CLASSES`, and
`device`
# !! are defined before this line.
study.optimize(
    lambda trial: objective(
        trial
    ),
    n_trials=n_trials
)
```

```
[I 2025-11-03 12:00:02,588] A new study created in memory with name:
no-name-dc2bc39b-e6bb-4dde-8c55-177a64dbea5a

Starting Optuna study with 10 trials...

[I 2025-11-03 12:03:38,054] Trial 0 finished with value:
0.860215053763441 and parameters: {'lr': 0.0003442262076311292,
'optimizer': 'Adam', 'batch_size': 16, 'out_channels_1': 32,
'out_channels_2': 128, 'dropout_rate': 0.20914902363865792,
'fc_features': 256}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 20 epochs.

Training complete in 3m 35s
Best Val Acc: 0.8602
```

```
[I 2025-11-03 12:06:20,630] Trial 1 finished with value:
0.8225806451612904 and parameters: {'lr': 0.0018202552734852416,
'optimizer': 'RMSprop', 'batch_size': 64, 'out_channels_1': 16,
'out_channels_2': 128, 'dropout_rate': 0.19285250048836633,
'fc_features': 64}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 15 epochs.

Training complete in 2m 43s
Best Val Acc: 0.8226

[I 2025-11-03 12:07:28,173] Trial 2 finished with value:
0.6021505376344086 and parameters: {'lr': 0.00800746458867836,
'optimizer': 'RMSprop', 'batch_size': 32, 'out_channels_1': 32,
'out_channels_2': 128, 'dropout_rate': 0.28183738458506136,
'fc_features': 128}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 6 epochs.

Training complete in 1m 8s
Best Val Acc: 0.6022

[I 2025-11-03 12:10:29,980] Trial 3 finished with value:
0.6666666666666667 and parameters: {'lr': 0.0007710553503685784,
'optimizer': 'SGD', 'batch_size': 32, 'out_channels_1': 32,
'out_channels_2': 128, 'dropout_rate': 0.4050823356104938,
'fc_features': 256}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 16 epochs.

Training complete in 3m 2s
Best Val Acc: 0.6667

[I 2025-11-03 12:13:43,460] Trial 4 finished with value:
0.7365591397849462 and parameters: {'lr': 4.901696548290351e-05,
'optimizer': 'RMSprop', 'batch_size': 64, 'out_channels_1': 16,
'out_channels_2': 64, 'dropout_rate': 0.1433708352947139,
'fc_features': 64}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 20 epochs.

Training complete in 3m 13s
Best Val Acc: 0.7366

[I 2025-11-03 12:16:04,598] Trial 5 finished with value:
0.8387096774193549 and parameters: {'lr': 0.0011386602503004424,
'optimizer': 'Adam', 'batch_size': 32, 'out_channels_1': 32,
'out_channels_2': 64, 'dropout_rate': 0.208688260883726,
'fc_features': 64}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 14 epochs.
```

```
Training complete in 2m 21s
Best Val Acc: 0.8387

[I 2025-11-03 12:16:06,018] Trial 6 finished with value: 0.0 and
parameters: {'lr': 0.0021614895615151286, 'optimizer': 'SGD',
'batch_size': 64, 'out_channels_1': 64, 'out_channels_2': 128,
'dropout_rate': 0.42115817677166545, 'fc_features': 64}. Best is trial
0 with value: 0.860215053763441.

Trial failed with error: CUDA out of memory. Tried to allocate 2.25
GiB. GPU 0 has a total capacity of 15.89 GiB of which 183.12 MiB is
free. Process 3090 has 15.71 GiB memory in use. Of the allocated
memory 11.47 GiB is allocated by PyTorch, and 3.94 GiB is reserved by
PyTorch but unallocated. If reserved but unallocated memory is large
try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid
fragmentation.  See documentation for Memory Management
(https://pytorch.org/docs/stable/notes/cuda.html#environment-
variables)

[I 2025-11-03 12:18:26,412] Trial 7 finished with value:
0.8225806451612904 and parameters: {'lr': 0.0004895861599810468,
'optimizer': 'RMSprop', 'batch_size': 32, 'out_channels_1': 64,
'out_channels_2': 64, 'dropout_rate': 0.1121645070140152,
'fc_features': 256}. Best is trial 0 with value: 0.860215053763441.

Early stopping triggered after 12 epochs.

Training complete in 2m 20s
Best Val Acc: 0.8226

[I 2025-11-03 12:18:27,645] Trial 8 finished with value: 0.0 and
parameters: {'lr': 7.820438429625042e-05, 'optimizer': 'RMSprop',
'batch_size': 64, 'out_channels_1': 64, 'out_channels_2': 64,
'dropout_rate': 0.48169019167467575, 'fc_features': 64}. Best is trial
0 with value: 0.860215053763441.

Trial failed with error: CUDA out of memory. Tried to allocate 2.25
GiB. GPU 0 has a total capacity of 15.89 GiB of which 177.12 MiB is
free. Process 3090 has 15.71 GiB memory in use. Of the allocated
memory 12.17 GiB is allocated by PyTorch, and 3.24 GiB is reserved by
PyTorch but unallocated. If reserved but unallocated memory is large
try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid
fragmentation.  See documentation for Memory Management
(https://pytorch.org/docs/stable/notes/cuda.html#environment-
variables)

[I 2025-11-03 12:21:36,267] Trial 9 finished with value:
0.6720430107526882 and parameters: {'lr': 1.0773800385711486e-05,
'optimizer': 'SGD', 'batch_size': 16, 'out_channels_1': 64,
'out_channels_2': 64, 'dropout_rate': 0.3727204872439678,
'fc_features': 256}. Best is trial 0 with value: 0.860215053763441.
```

```
Early stopping triggered after 17 epochs.

Training complete in 3m 8s
Best Val Acc: 0.6720

# --- 4. Get Best Results ---
print("\n--- Optuna Search Complete ---")
print(f"Best Validation Accuracy: {study.best_value:.4f}")
print(f"Best Hyperparameters: {study.best_params}")


--- Optuna Search Complete ---
Best Validation Accuracy: 0.8602
Best Hyperparameters: {'lr': 0.0003442262076311292, 'optimizer':
'Adam', 'batch_size': 16, 'out_channels_1': 32, 'out_channels_2': 128,
'dropout_rate': 0.20914902363865792, 'fc_features': 256}

best_params = study.best_params
batch_size = best_params['batch_size']

train_loader = DataLoader(train_dataset_, batch_size=batch_size,
shuffle=True)
val_loader = DataLoader(val_dataset_, batch_size=batch_size,
shuffle=False)
test_loader = DataLoader(test_dataset_, batch_size=batch_size,
shuffle=False)

model = TunableCNN(in_channels, num_classes, trial=None,
**best_params).to(device)

# Create Optimizer

optimizer_class = getattr(optim, best_params['optimizer'])
optimizer = optimizer_class(model.parameters(), lr=best_params['lr'])

# Loss function
criterion = nn.CrossEntropyLoss()

final_model, history = train_model(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    epochs=100,  # Train for a full run
    patience=10  # With reasonable patience
)

# --- 5. Evaluate on Test Set ---
print("\nEvaluating Final Model on Test Set...")
```

```
print("-------------------------------------")
abc = evaluate_model(
    model=final_model,
    loader=test_loader,
    criterion=criterion,
    device=device
)
```

```
Epoch 1/100 | Train Loss: 0.6790 Acc: 0.6020 | Val Loss: 0.5301 Acc:
0.7204
Epoch 2/100 | Train Loss: 0.6125 Acc: 0.6834 | Val Loss: 0.5411 Acc:
0.7043
Epoch 3/100 | Train Loss: 0.5875 Acc: 0.7113 | Val Loss: 0.5092 Acc:
0.7419
Epoch 4/100 | Train Loss: 0.5479 Acc: 0.7358 | Val Loss: 0.5371 Acc:
0.7204
Epoch 5/100 | Train Loss: 0.5264 Acc: 0.7503 | Val Loss: 0.4780 Acc:
0.7634
Epoch 6/100 | Train Loss: 0.5107 Acc: 0.7447 | Val Loss: 0.4603 Acc:
0.7796
Epoch 7/100 | Train Loss: 0.4971 Acc: 0.7804 | Val Loss: 0.4685 Acc:
0.7688
Epoch 8/100 | Train Loss: 0.4884 Acc: 0.7726 | Val Loss: 0.4845 Acc:
0.7957
Epoch 9/100 | Train Loss: 0.4850 Acc: 0.7904 | Val Loss: 0.4171 Acc:
0.8280
Epoch 10/100 | Train Loss: 0.4953 Acc: 0.7837 | Val Loss: 0.6143 Acc:
0.7043
Epoch 11/100 | Train Loss: 0.4844 Acc: 0.7670 | Val Loss: 0.5406 Acc:
0.7527
Epoch 12/100 | Train Loss: 0.4529 Acc: 0.7815 | Val Loss: 0.4270 Acc:
0.8226
Epoch 13/100 | Train Loss: 0.4278 Acc: 0.7993 | Val Loss: 0.4677 Acc:
0.8065
Epoch 14/100 | Train Loss: 0.4169 Acc: 0.8216 | Val Loss: 0.4217 Acc:
0.8280
Epoch 15/100 | Train Loss: 0.4051 Acc: 0.8138 | Val Loss: 0.4062 Acc:
0.8387
Epoch 16/100 | Train Loss: 0.3967 Acc: 0.8317 | Val Loss: 0.3802 Acc:
0.8441
Epoch 17/100 | Train Loss: 0.4042 Acc: 0.8161 | Val Loss: 0.3812 Acc:
0.8333
Epoch 18/100 | Train Loss: 0.3991 Acc: 0.8328 | Val Loss: 0.4056 Acc:
0.8280
Epoch 19/100 | Train Loss: 0.3780 Acc: 0.8283 | Val Loss: 0.3872 Acc:
0.8602
Epoch 20/100 | Train Loss: 0.3740 Acc: 0.8372 | Val Loss: 0.6214 Acc:
0.7903
Epoch 21/100 | Train Loss: 0.3593 Acc: 0.8339 | Val Loss: 0.3814 Acc:
0.8495
```

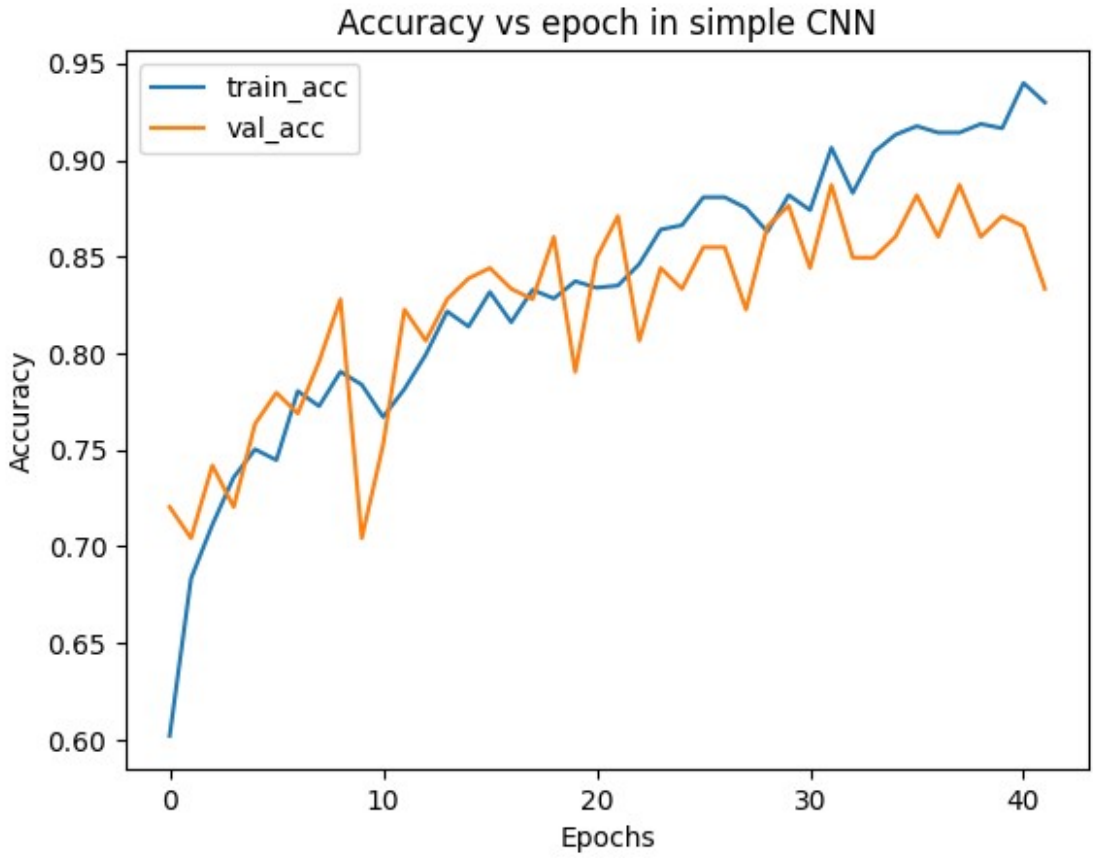
```
Epoch 22/100 | Train Loss: 0.3648 Acc: 0.8350 | Val Loss: 0.3594 Acc:
0.8710
Epoch 23/100 | Train Loss: 0.3540 Acc: 0.8462 | Val Loss: 0.4541 Acc:
0.8065
Epoch 24/100 | Train Loss: 0.3460 Acc: 0.8640 | Val Loss: 0.4732 Acc:
0.8441
Epoch 25/100 | Train Loss: 0.3233 Acc: 0.8662 | Val Loss: 0.4150 Acc:
0.8333
Epoch 26/100 | Train Loss: 0.3174 Acc: 0.8807 | Val Loss: 0.4006 Acc:
0.8548
Epoch 27/100 | Train Loss: 0.3112 Acc: 0.8807 | Val Loss: 0.4065 Acc:
0.8548
Epoch 28/100 | Train Loss: 0.3141 Acc: 0.8751 | Val Loss: 0.5727 Acc:
0.8226
Epoch 29/100 | Train Loss: 0.3223 Acc: 0.8629 | Val Loss: 0.3991 Acc:
0.8656
Epoch 30/100 | Train Loss: 0.2906 Acc: 0.8818 | Val Loss: 0.3791 Acc:
0.8763
Epoch 31/100 | Train Loss: 0.3010 Acc: 0.8740 | Val Loss: 0.4949 Acc:
0.8441
Epoch 32/100 | Train Loss: 0.2714 Acc: 0.9064 | Val Loss: 0.4523 Acc:
0.8871
Epoch 33/100 | Train Loss: 0.3032 Acc: 0.8829 | Val Loss: 0.4957 Acc:
0.8495
Epoch 34/100 | Train Loss: 0.2727 Acc: 0.9041 | Val Loss: 0.4715 Acc:
0.8495
Epoch 35/100 | Train Loss: 0.2584 Acc: 0.9130 | Val Loss: 0.5040 Acc:
0.8602
Epoch 36/100 | Train Loss: 0.2478 Acc: 0.9175 | Val Loss: 0.4544 Acc:
0.8817
Epoch 37/100 | Train Loss: 0.2469 Acc: 0.9142 | Val Loss: 0.4835 Acc:
0.8602
Epoch 38/100 | Train Loss: 0.2287 Acc: 0.9142 | Val Loss: 0.5363 Acc:
0.8871
Epoch 39/100 | Train Loss: 0.2403 Acc: 0.9186 | Val Loss: 0.4272 Acc:
0.8602
Epoch 40/100 | Train Loss: 0.2144 Acc: 0.9164 | Val Loss: 0.5246 Acc:
0.8710
Epoch 41/100 | Train Loss: 0.1925 Acc: 0.9398 | Val Loss: 0.4829 Acc:
0.8656
Epoch 42/100 | Train Loss: 0.1998 Acc: 0.9298 | Val Loss: 0.6214 Acc:
0.8333
Early stopping triggered after 42 epochs.

Training complete in 7m 17s
Best Val Acc: 0.8871

Evaluating Final Model on Test Set...
--------------------------------------
```

```
Loss: 0.8980
Accuracy: 0.7121
Precision: 0.7123
Recall: 0.7121
F1-score: 0.7121
```

```
plt.plot(history['train_acc'], label="train_acc")
plt.plot(history['val_acc'], label="val_acc")
plt.title("Accuracy vs epoch in simple CNN")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```




```
torch.save(final_model, "simple_final_model.pth")
```

Transfer Learning

```
weights = models.ViT_B_16_Weights.IMAGENET1K_SWAG_E2E_V1
vit_base = models.vit_b_16(weights=weights)
```

```
# 2. To create the feature extractor, we replace the final
classification 'heads'
#    with an Identity layer, so the model returns the feature vector.
#    The feature vector for ViT-B-16 is 768-dimensional.
vit_base.heads = nn.Identity()

vit_base = vit_base.to(device)
vit_base.eval() # Set to evaluation mode

print("ViT feature extractor created.")
```

```
Downloading: "https://download.pytorch.org/models/vit_b_16_swag-
9ac1b537.pth" to /root/.cache/torch/hub/checkpoints/vit_b_16_swag-
9ac1b537.pth
100%|██████████| 331M/331M [00:01<00:00, 229MB/s]

ViT feature extractor created.
```

```
# --- 5. Train Final Model and Evaluate ---
print("\n--- Training and evaluating final best model ---")
best_params = study.best_params

# Setup final components
train_loader = DataLoader(train_dataset_, batch_size=batch_size,
shuffle=True)
        val_loader = DataLoader(val_dataset_, batch_size=batch_size,
shuffle=False)
        test_loader = DataLoader(test_dataset_, batch_size=batch_size,
shuffle=False)

final_model = FeatureClassifier(
    input_features=input_dim,
    num_classes=len(classes),
    hidden_dim=best_params['hidden_dim'],
    dropout_rate=best_params['dropout_rate']
).to(device)

model, history = train_model(
            model=model,
            train_loader=train_loader,
            val_loader=val_loader,
            criterion=criterion,
            optimizer=optimizer,
            device=device,
            epochs=epochs,
            patience=patience,
            verbose=0  # Set to 0 to keep Optuna's output clean
        )

print("\n--- Evaluating best model on TEST set ---")
```

```
ab_testing = evaluate_model(final_model, test_feat_loader, criterion,
device)

print("\n--- Evaluating best model on TRAINING set ---")
ab_training = evaluate_model(final_model, train_feat_loader,
criterion, device)

batch_size=20

train_loader = DataLoader(train_dataset_, batch_size=batch_size,
shuffle=True, pin_memory=True)
val_loader = DataLoader(val_dataset_, batch_size=batch_size,
shuffle=False, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
shuffle=False, pin_memory=True)

def rgb_loader(path):
    with open(path, 'rb') as f:
        img = Image.open(f)
        return img.convert('RGB')

def extract_features_torch(loader, model, device):
    """
    Extracts features from a dataloader using a model.
    """
    features = []
    all_labels = []

    with torch.no_grad():
        for inputs, labels in tqdm(loader):
            inputs = inputs.to(device)

            # Pass through the convolutional base
            feats = model(inputs)

            # Move features to CPU and store
            features.append(feats.cpu())
            all_labels.append(labels.cpu())

    # Concatenate all batches
    features = torch.cat(features, dim=0)
    all_labels = torch.cat(all_labels, dim=0)

    return features, all_labels

# Define the simple classifier
class FeatureClassifier(nn.Module):
    def __init__(self, input_features, num_classes=14):
        super(FeatureClassifier, self).__init__()
        self.classifier = nn.Sequential(
```

```
            nn.Linear(input_features, 256),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        return self.classifier(x)

print("Extracting training features...")
train_features, train_labels = extract_features_torch(train_loader,
vit_base, device)
print("Extracting validation features...")
validation_features, validation_labels =
extract_features_torch(val_loader, vit_base, device)
print("Extracting test features...")
test_features, test_labels = extract_features_torch(test_loader,
vit_base, device)

print(f"Train features shape: {train_features.shape}, Train labels
shape: {train_labels.shape}")
print(f"Val features shape: {validation_features.shape}, Val labels
shape: {validation_labels.shape}")
```

```
Extracting training features...

100%|██████████| 45/45 [00:22<00:00,  2.01it/s]

Extracting validation features...

100%|██████████| 10/10 [00:04<00:00,  2.17it/s]

Extracting test features...

100%|██████████| 2/2 [00:00<00:00,  3.64it/s]

Train features shape: torch.Size([897, 768]), Train labels shape:
torch.Size([897])
Val features shape: torch.Size([186, 768]), Val labels shape:
torch.Size([186])
```

```
# Create new TensorDatasets and DataLoaders
# We can use the features directly, no flattening needed.
batch_size = 20
train_feat_dataset = TensorDataset(train_features, train_labels)
val_feat_dataset = TensorDataset(validation_features,
validation_labels)
test_feat_dataset = TensorDataset(test_features, test_labels)

train_feat_loader = DataLoader(train_feat_dataset,
```

```
batch_size=batch_size, shuffle=True)
val_feat_loader = DataLoader(val_feat_dataset, batch_size=batch_size,
shuffle=False)
test_feat_loader = DataLoader(test_feat_dataset,
batch_size=batch_size, shuffle=False)
classes = train_dataset.classes

# %%
# Define the simple classifier
# The input dimension for ViT-B-16 is 768
input_dim = 768
model_feat_classifier = FeatureClassifier(input_dim,
num_classes=len(classes)).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.RMSprop(model_feat_classifier.parameters(), lr=2e-5)

# Train the classifier

epochs=100
model_feat_classifier, history_feat = train_model(
    model_feat_classifier, train_feat_loader, val_feat_loader,
    criterion, optimizer, device,
    epochs=epochs, patience=20,
    verbose=0
)
print("\nTesting")
ab_testing = evaluate_model(model_feat_classifier, test_feat_loader,
criterion, device)

print("\nTraining")
ab_training = evaluate_model(model_feat_classifier, train_feat_loader,
criterion, device)

plt.plot(history_feat['train_acc'])
plt.plot(history_feat['val_acc'])

torch.save(model_feat_classifier, "final_model.pth")
```

Hyper parameter Tuning

```
# MODIFIED FeatureClassifier
class FeatureClassifier(nn.Module):
    def __init__(self, input_features, num_classes, hidden_dim=256,
dropout_rate=0.1):
        """
        Initialize the classifier.
        Args:
```

```
            input_features (int): Dimension of the input features
(e.g., 768 for ViT-B-16)
            num_classes (int): Number of output classes
            hidden_dim (int): Number of neurons in the hidden layer
            dropout_rate (float): Dropout probability
        """
        super(FeatureClassifier, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(input_features, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, num_classes)
        )

    def forward(self, x):
        return self.classifier(x)

import optuna
import copy

# --- 1. Define Data and Constants (Same as before) ---
train_feat_dataset = TensorDataset(train_features, train_labels)
val_feat_dataset = TensorDataset(validation_features,
validation_labels)
test_feat_dataset = TensorDataset(test_features, test_labels)

classes = train_dataset.classes
input_dim = 768
criterion = nn.CrossEntropyLoss()
epochs = 100
patience = 20

# --- 2. Define the Objective Function for Optuna ---
# This function trains one model and returns its score
def objective(trial):
    # --- A. Suggest Hyperparameters ---
    params = {
        'lr': trial.suggest_float('lr', 1e-6, 1e-3, log=True),
        'batch_size': trial.suggest_categorical('batch_size', [16, 32,
64]),
        'hidden_dim': trial.suggest_categorical('hidden_dim', [128,
256, 512]),
        'dropout_rate': trial.suggest_float('dropout_rate', 0.1, 0.5),
        'optimizer_type': trial.suggest_categorical('optimizer_type',
['Adam', 'RMSprop'])
    }

    # --- B. Setup components based on suggested params ---
    train_feat_loader = DataLoader(train_feat_dataset,
batch_size=params['batch_size'], shuffle=True)
```

```
    val_feat_loader = DataLoader(val_feat_dataset,
batch_size=params['batch_size'], shuffle=False)

    model_feat_classifier = FeatureClassifier(
        input_features=input_dim,
        num_classes=len(classes),
        hidden_dim=params['hidden_dim'],
        dropout_rate=params['dropout_rate']
    ).to(device)

    if params['optimizer_type'] == 'Adam':
        optimizer = optim.Adam(model_feat_classifier.parameters(),
lr=params['lr'])
    else:
        optimizer = optim.RMSprop(model_feat_classifier.parameters(),
lr=params['lr'])

    # --- C. Train the model ---
    model, history = train_model(
        model_feat_classifier, train_feat_loader, val_feat_loader,
        criterion, optimizer, device,
        epochs=epochs, patience=patience,
        verbose=0 # Keep it quiet
    )

    # --- D. Return the best validation accuracy ---
    best_val_acc = max(history['val_acc'])
    return best_val_acc

# --- 3. Run the Optuna Study ---
# We want to 'maximize' the validation accuracy
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)  # Run 50 trials


# --- 4. Get Best Results ---
print("\n--- Optuna Search Complete ---")
print(f"Best Validation Accuracy: {study.best_value:.4f}")
print(f"Best Hyperparameters: {study.best_params}")


# --- 5. Train Final Model and Evaluate ---
print("\n--- Training and evaluating final best model ---")
best_params = study.best_params

# Setup final components
train_feat_loader = DataLoader(train_feat_dataset,
batch_size=best_params['batch_size'], shuffle=True)
val_feat_loader = DataLoader(val_feat_dataset,
batch_size=best_params['batch_size'], shuffle=False)
```

```
test_feat_loader = DataLoader(test_feat_dataset,
batch_size=best_params['batch_size'], shuffle=False)

final_model = FeatureClassifier(
    input_features=input_dim,
    num_classes=len(classes),
    hidden_dim=best_params['hidden_dim'],
    dropout_rate=best_params['dropout_rate']
).to(device)

if best_params['optimizer_type'] == 'Adam':
    optimizer = optim.Adam(final_model.parameters(),
lr=best_params['lr'])
else:
    optimizer = optim.RMSprop(final_model.parameters(),
lr=best_params['lr'],weight_decay=1e-4)

# Train the final model (you can use verbose=1 now)
final_model, history = train_model(
    final_model, train_feat_loader, val_feat_loader,
    criterion, optimizer, device,
    epochs=epochs, patience=patience,
    verbose=1
)

print("\n--- Evaluating best model on TEST set ---")
ab_testing = evaluate_model(final_model, test_feat_loader, criterion,
device)

print("\n--- Evaluating best model on TRAINING set ---")
ab_training = evaluate_model(final_model, train_feat_loader,
criterion, device)


# --- 5. Train Final Model and Evaluate ---
print("\n--- Training and evaluating final best model ---")
best_params = study.best_params

# Setup final components
train_feat_loader = DataLoader(train_feat_dataset,
batch_size=best_params['batch_size'], shuffle=True)
val_feat_loader = DataLoader(val_feat_dataset,
batch_size=best_params['batch_size'], shuffle=False)
test_feat_loader = DataLoader(test_feat_dataset,
batch_size=best_params['batch_size'], shuffle=False)

final_model = FeatureClassifier(
    input_features=input_dim,
    num_classes=len(classes),
    hidden_dim=128,
```

```
    dropout_rate=best_params['dropout_rate']
).to(device)

if best_params['optimizer_type'] == 'Adam':
    optimizer = optim.Adam(final_model.parameters(),
lr=best_params['lr'])
else:
    optimizer = optim.RMSprop(final_model.parameters(),
lr=best_params['lr'],weight_decay=1e-4)

# Train the final model (you can use verbose=1 now)
final_model, history = train_model(
    final_model, train_feat_loader, val_feat_loader,
    criterion, optimizer, device,
    epochs=epochs, patience=patience,
    verbose=0
)

print("\n--- Evaluating best model on TEST set ---")
ab_testing = evaluate_model(final_model, test_feat_loader, criterion,
device)

print("\n--- Evaluating best model on TRAINING set ---")
ab_training = evaluate_model(final_model, train_feat_loader,
criterion, device)

plt.plot(history['train_acc'], label="train_acc")
plt.plot(history['val_acc'], label="val_acc")
plt.legend()
plt.show()

torch.save(final_model, "final_model.pth")
```